Kotlin Backend Developer.Professional

Проверить, идет ли запись

Меня хорошо видно && слышно?

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-ГГГГ-ММ

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Создаем внутренние модели и мапперы

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП	DSL	
Модуль 2: Расширенные возможности Kotlin	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle	
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	DevOps	Архитектура		
Модуль 4: Разработка API	OpenAPI	Модели и мапперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози- торий	Конкурен- ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль- тация	Защита			

Маршрут вебинара

- Транспортный модуль
- Внутренние модели
- Мапперы
- Рефлексия

Цели вебинара

После занятия вы:

1. Сможете создавать внутренние модели и контекст
2. Сможете разрабатывать маппинги

Смысл

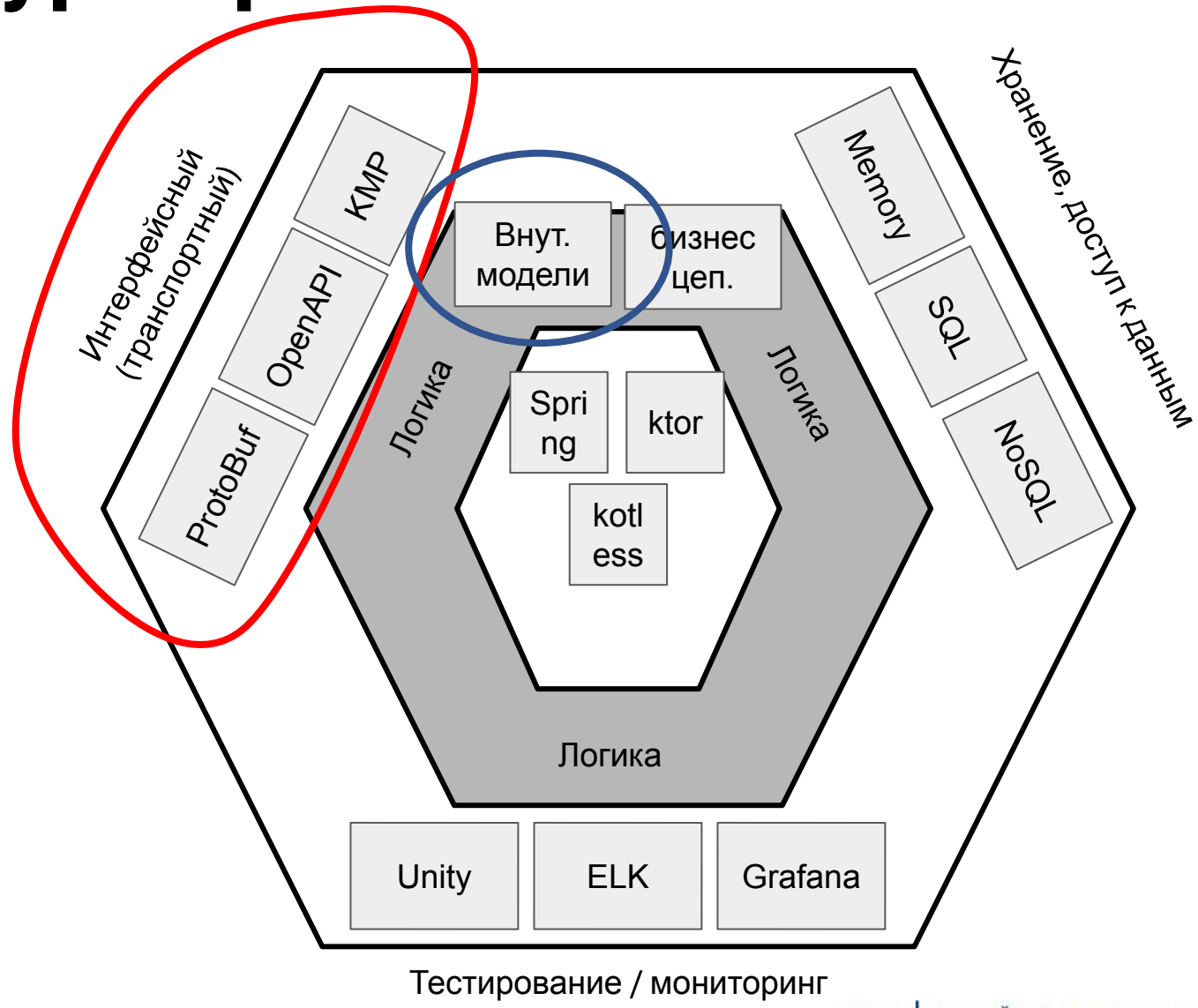
Зачем вам это уметь

1. Освоить базовые элементы DDD и принципы формирования внутренних моделей
2. Контекст будет использоваться во всем приложении в качестве связующего звена между всеми модулями
3. DDD обеспечивает изоляцию компонентов приложения, что дает гибкость при управлении изменениями

API-модели: что и как?

Целевая архитектура приложения

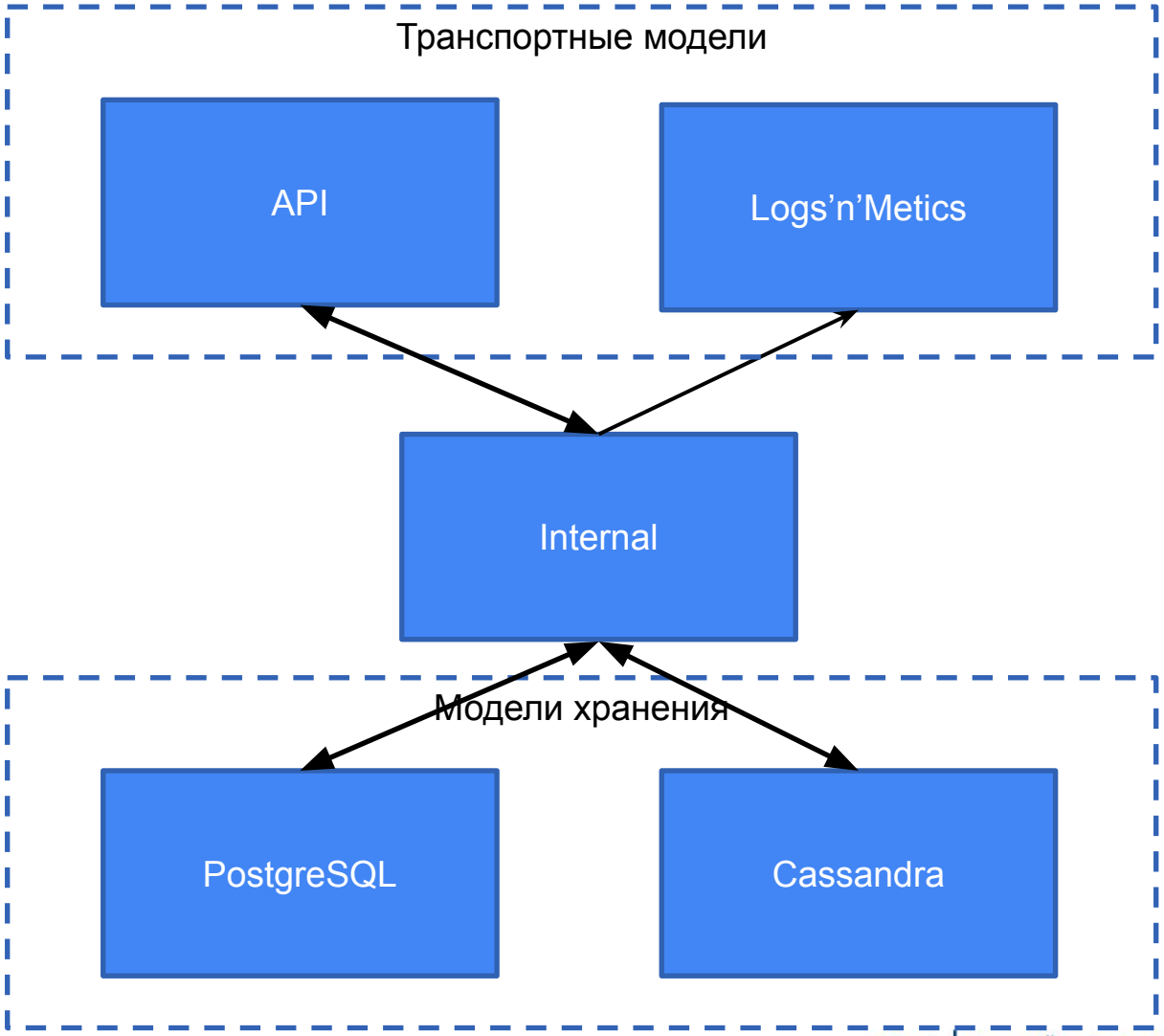
- Разработка серверного приложения начинается с API-моделей
- API-модели ориентированы на требования потребителя (фронтенд или клиентский сервис)
- Согласованы на уровне спецификации
- Сложный процесс изменения
- Одно приложение может поддерживать одновременно разные API (разные версии, разные клиенты)
- Возможна поддержка разных транспортных протоколов (REST, Websocket, Kafka, etc)



Модели в приложении

Для разных задач используются разные модели, оптимизированные под задачу:

- API
- Logs’n’Metrics
- Internal
- Store



Транспортные модели

Назначение – пересылка данных

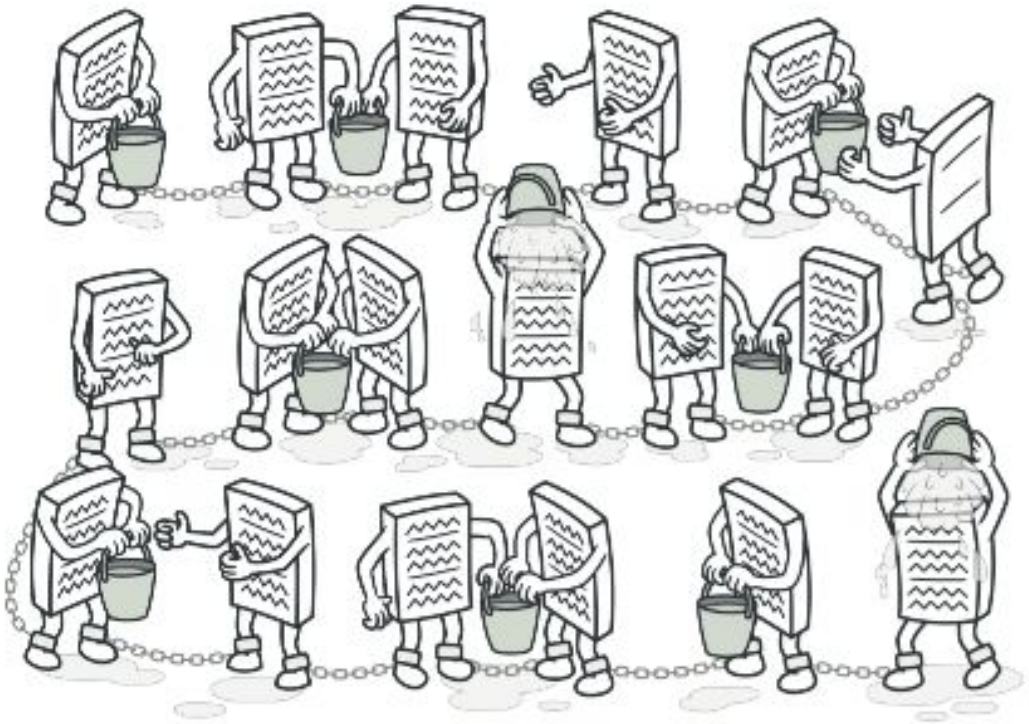
Особенности:

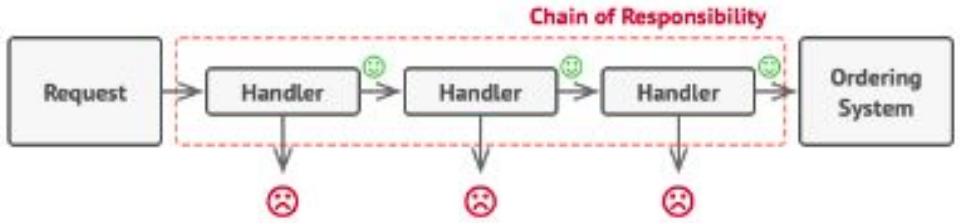
- Статичные - ограничиваются контрактом
- Компактные, но достаточные (полные)
- Содержат данные, необходимые потребителю, и служебные данные
- Небезопасные, должны быть nullable, требуют валидации
- Сериализуемые, помечаются @Serializable (kotlinx.serializer)

Принципы целевой архитектуры (Контекст)

Логика приложения & Context & Chain of Responsibility:

- Отдельный модуль для бизнес-логики
- Логика на шаблоне проектирования Chain of Responsibility (Цепочка обязанностей)
- Context - контейнер для всех внутренних моделей, флагов, статусов и переменных
- Контекст связывает различные домены внутри одного микросервиса через мапперы





Контекст

1. Содержит текущее состояние, настройки и параметры обработки запроса
2. Два подхода:
   a. Свойства мутабельны, могут меняться в ходе обработки
   b. Свойства иммутабельны, изменение через копирование
3. Свойства имеют non-Null значения по умолчанию
4. Ничего не знает о транспортных моделях и моделях хранения

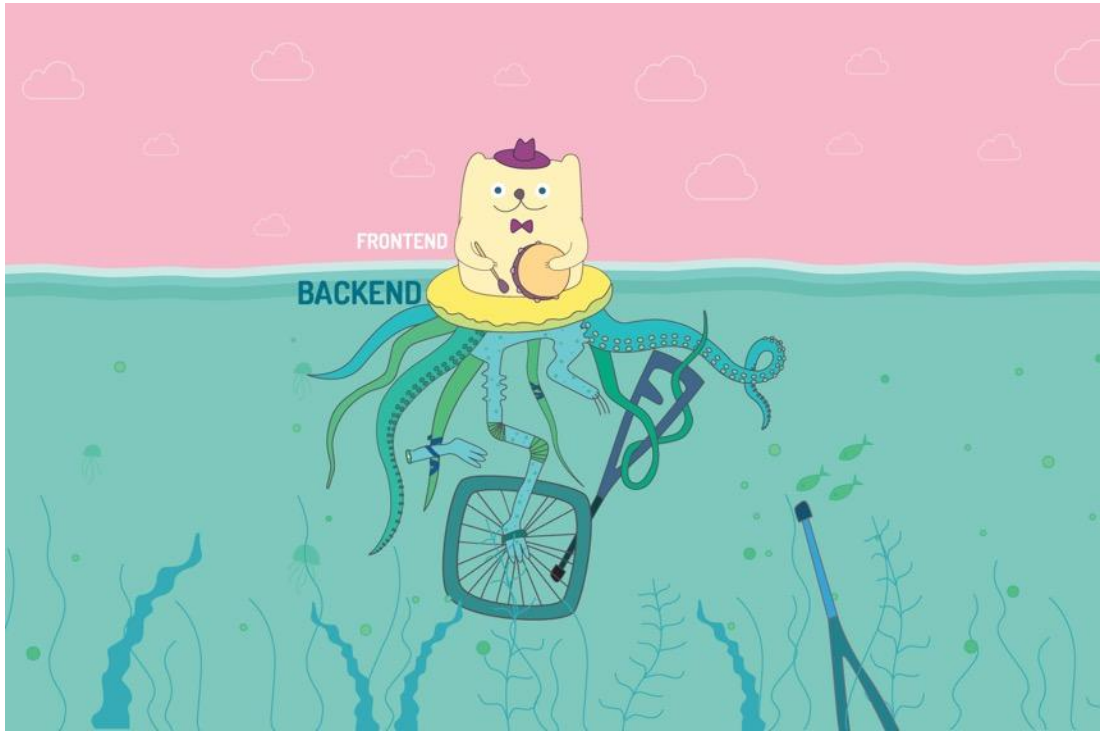
```
data class MpBeContext(
    var timeStarted: Instant = Instant.MIN,
    var responseId: String = "",
    var onRequest: String = "",
    var status: MpBeContextStatus = MpBeContextStatus.NONE,
    var errors: MutableList<IMpError> = mutableListOf(),
    var stubCase: MpStubCase = MpStubCase.NONE,
    var workMode: MpWorkMode = MpWorkMode.DEFAULT,
    var requestDemand: MpDemandModel = MpDemandModel.NONE,
    var responseDemand: MpDemandModel = MpDemandModel.NONE,
    var demandRepo: IDemandRepository = IDemandRepository.NONE,
)
```

Внутренние модели

Назначение – связывание различных компонентов внутри нашей программы на уровне бизнес-логики

Особенности:

- Динамичные – могут меняться в ходе разработки
- Могут быть избыточными
- DTO: следует выносить логику в отдельные классы
- Следует избегать nullable полей
- Поля имеют значения по умолчанию



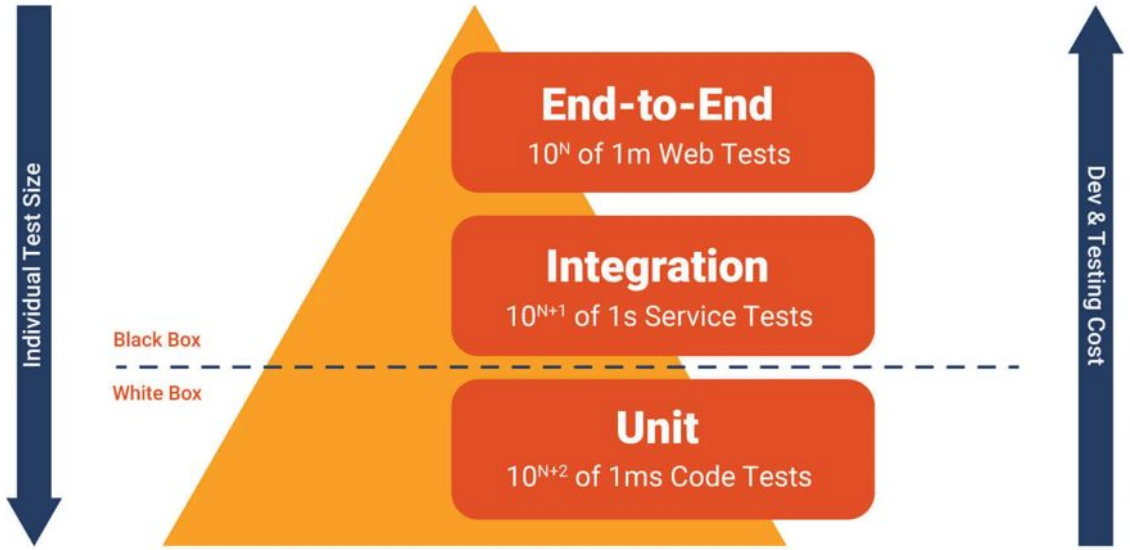
Маппинг

- Связывает внутренние модели с транспортными
- Реализует ЛОГИКУ конвертации одних моделей в другие
- Может использовать контекст
- Может располагаться в отдельном модуле от внутренних и транспортных моделей

Дополнение к архитектуре

Требования:

1. Различные уровни тестирования:
   a. модульное
   b. интеграционное
   c. сквозное
2. Инфраструктура для генерации метрик
3. Специальный режим тестирования в приложении:
   a. объект debug в запросах и ответах
4. Каждый запрос содержит идентификатор для отслеживания всего пути его через систему
   a. По стандарту в заголовках HTTP



LIVE

Вопросы

1. Как проектируются транспортные модели?
2. Чем транспортные модели отличаются от внутренних?
3. Зачем нужен маппинг транспортных моделей?

5 мин

Тезисы

Подведем итоги

1. API-модели содержат данные для потребителя и служебные данные
2. API-модели выносятся в отдельный модуль, либо создаются в модуле транспортных моделей
3. Внутренние модели служат для связывания данных внутри приложения, могут быть избыточными
4. Контекст служит контейнером для текущего состояния и используется в цепочке обработки запроса
5. Маппинг нужен для преобразований между транспортными моделями и внутренними

Слайд с домашним заданием

1. Сформировать транспортные модели для вашего выпускного проекта
2. Сделать тест на сериализацию/десериализацию
3. Реализовать внутренние модели
4. Обеспечить мапинг между транспортными и внутренними моделями

Сроки выполнения: до завершения Модуля 4

Рефлексия

Проверка достижения целей

После занятия вы:

1. Сможете создавать внутренние модели и контекст
2. Сможете разрабатывать маппинги

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Список материалов для изучения

1. https://kotlinlang.org/
2. https://gradle.org/
3. https://github.com/otuskotlin/ГГГГММ-ok-marketplace
4. https://ru.wikipedia.org/wiki/Предметно-ориентированное_проектирование
5. https://ru.wikipedia.org/wiki/Цепочка_обязанностей

Следующий вебинар

Модуль 4: Разработка API

Тестирование проекта

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары